

Лабораторная работа №13

Программирование в командном процессоре ОС UNIX. Ветвления
и циклы

true

Лабораторная работа №13. Программирование в командном
процессоре ОС UNIX. Ветвления и циклы

Хайдаров Олим. НБИбд-02-25

Содержание

Цель работы

Выполнение лабораторной работы

2.0 Использование getopt и grep

2.1 Взаимодействие Си и Bash

2.2 Циклы и управление файлами

2.3 Архивация с помощью find

Выводы

Контрольные вопросы

Список литературы

Лабораторная работа №13.

Программирование в командном процессоре ОС UNIX. Ветвления и циклы

Хайдаров Олим. НБИбд-02-25

Содержание

1. Цель работы
 2. Выполнение лабораторной работы
 - 2.0 Использование getopts и grep
 - 2.1 Взаимодействие Си и Bash
 - 2.2 Циклы и управление файлами
 - 2.3 Архивация с помощью find
 3. Вывод
 4. Контрольные вопросы
 5. Список литературы
-

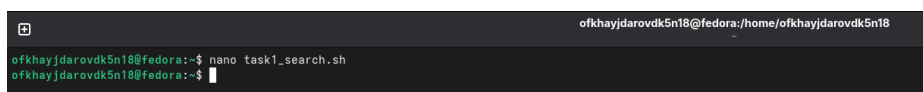
Цель работы

Изучить основы программирования в оболочке ОС UNIX.
Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Выполнение лабораторной работы

2.0 Использование getopts и grep

Условие: Написать командный файл, который анализирует командную строку с ключами -i, -o, -p, -C, -n и ищет в указанном файле нужные строки.



```
ofkhayjdarovdk5n18@fedora:~/home/ofkhayjdarovdk5n18
ofkhayjdarovdk5n18@fedora:~$ nano task1_search.sh
ofkhayjdarovdk5n18@fedora:~$
```

Создание файла

Пишем код Командного файла:

```
#!/bin/bash

# =====
# Лабораторная работа №11, Задание 1
# Поиск строк в файле с помощью getopt и grep
# =====

# Переменные для хранения параметров
INPUT_FILE=""
OUTPUT_FILE=""
PATTERN=""
CASE_SENSITIVE=""
LINE_NUMBERS=""

# Парсинг командной строки с помощью getopt
# i: -i требует аргумент (имя входного файла)
# o: -o требует аргумент (имя выходного файла)
# p: -p требует аргумент (шаблон поиска)
# C: -C не требует аргумента (регистрозависимый поиск)
# n: -n не требует аргумента (показывать номера строк)

while getopt "i:o:p:Cn" opt; do
    case $opt in
        i)
            INPUT_FILE="$OPTARG"
            echo "📁 Входной файл: $INPUT_FILE"
            ;;
        o)
            OUTPUT_FILE="$OPTARG"
            echo "📁 Выходной файл: $OUTPUT_FILE"
            ;;
        p)
            PATTERN="$OPTARG"
            echo "🔍 Шаблон поиска: $PATTERN"
            ;;
        C)
            CASE_SENSITIVE="-C"
            echo "🔍 Поиск с учётом регистра (ключ -C)"
            ;;
        n)
            LINE_NUMBERS="-n"
            echo "🔍 Показывать номера строк (ключ -n)"
            ;;
        \?)
            echo "❌ Ошибка: неверный ключ -$OPTARG"
            exit 1
    esac
done
```

Командный файл, 1-я часть

```

        echo "❌ Ошибка: неверный ключ -$OPTARG"
        exit 1
    ;;
:)
    echo "❌ Ошибка: ключ -$OPTARG требует аргумент"
    exit 1
    ;;
esac
done

# Проверка обязательных параметров
if [ -z "$INPUT_FILE" ] || [ -z "$OUTPUT_FILE" ] || [ -z "$PATTERN" ]; then
    echo ""
    echo "❌ Ошибка: не указаны все обязательные параметры!"
    echo ""
    echo "Использование:"
    echo " $0 -i <входной_файл> -o <выходной_файл> -r <шаблон> [-C] [-n]"
    echo ""
    echo "Параметры:"
    echo " -i файл      - файл для поиска"
    echo " -o файл      - файл для сохранения результатов"
    echo " -r шаблон    - текст для поиска"
    echo " -C           - учитывать регистр (по умолчанию не учитывается)"
    echo " -n           - показывать номера строк"
    exit 1
fi

# Проверка существования входного файла
if [ ! -f "$INPUT_FILE" ]; then
    echo "❌ Ошибка: файл '$INPUT_FILE' не найден!"
    exit 1
fi

echo ""
echo "=====
echo "🔍 Начинаю поиск..."
echo "=====

# Формируем флаги для grep
GREP_FLAGS=""

# Если не указан -C, то делаем поиск без учёта регистра (-i)
if [ -z "$CASE_SENSITIVE" ]; then
    GREP_FLAGS="-i"
fi

# Добавляем флаг номеров строк, если указан
if [ -n "$LINE_NUMBERS" ]; then

```

Командный файл, 2-я часть

```

echo 🚧 начинаем поиск...
echo "===== "

# Формируем флаги для grep
GREP_FLAGS=""

# Если не указан -C, то делаем поиск без учёта регистра (-i)
if [ -z "$CASE_SENSITIVE" ]; then
    GREP_FLAGS="-i"
fi

# Добавляем флаг номеров строк, если указан
if [ -n "$LINE_NUMBERS" ]; then
    GREP_FLAGS="$GREP_FLAGS -n"
fi

# Выполняем поиск и сохраняем результат
echo 🚧 Выполняем: grep $GREP_FLAGS '$PATTERN' '$INPUT_FILE' > '$OUTPUT_FILE'
grep $GREP_FLAGS "$PATTERN" "$INPUT_FILE" > "$OUTPUT_FILE"

# Проверяем результат
if [ $? -eq 0 ]; then
    echo 🟢 Поиск завершен успешно!
    echo ""

    # Считаем количество найденных строк
    COUNT=$(wc -l < "$OUTPUT_FILE")
    echo 📄 Найдено строк: $COUNT
    echo ""

    # Показываем результат
    if [ "$COUNT" -gt 0 ]; then
        echo 📄 Результаты поиска:
        echo "-----"
        cat "$OUTPUT_FILE"
        echo "-----"
    fi
else
    echo ⚠️ Ничего не найдено или произошла ошибка
    exit 1
fi

```

Командный файл, 3-я часть

- создаем тестовый файл - в нем будет набор строк с информацией, фрагмент которой мы будем в дальнейшем искать (помимо искомой, мы добавим строки с содержанием другой информации, для проверки работы программы) - это слово “Apple”

GNU nano 8.5	test_data.txt	Modified
Apple - это фрукт		
banana - жёлтый фрукт		
Apple iPhone - смартфон		
Cherry - вишня		
apple pie - яблочный пирог		
BANANA - заглавными буквами		

Тестовый файл

Далее мы делаем файл task1_search.sh исполняемым и запускаем процесс проверки работы:

```

ofkhayjdarovdk5n18@fedora:~$ chmod +x task1_search.sh
ofkhayjdarovdk5n18@fedora:~$ nano test_data.txt
ofkhayjdarovdk5n18@fedora:~$ ./task1_search.sh -i test_data.txt -o result.txt -p
"apple" -n
  Входной файл: test_data.txt
  Выходной файл: result.txt
  Шаблон поиска: apple
  Показывать номера строк (ключ -n)

=====
🚀 Начинаю поиск...
=====
📄 Выполняем: grep -i -n 'apple' 'test_data.txt' > 'result.txt'
✅ Поиск завершён успешно!

📊 Найдено строк: 3

📋 Результаты поиска:
-----
1:Apple - это фрукт
3:Apple iPhone - смартфон
5:apple pie - яблочный пирог
-----
ofkhayjdarovdk5n18@fedora:~$

```

Проверка работы

Также если мы введем файл без искомого слова, которое мы ищем и без пути, то нам выдается следующее:

```

ofkhayjdarovdk5n18@fedora:~$ ./task1_search.sh -i test_data.txt
  Входной файл: test_data.txt

❌ Ошибка: не указаны все обязательные параметры!

Использование:
./task1_search.sh -i <входной_файл> -o <выходной_файл> -p <шаблон> [-C] [-n]

Параметры:
-i файл    - файл для поиска
-o файл    - файл для сохранения результатов
-p шаблон  - текст для поиска
-C         - учитывать регистр (по умолчанию не учитывается)
-n         - показывать номера строк
ofkhayjdarovdk5n18@fedora:~$

```

Тест файла

После выдётся опции параметров ввода – снизу сообщения об ошибки – с целью, чтобы пользователь специфицировал запрос на основе предложенных вариантов ввода

Также мы далее смотрим, что было записано в реестр-файл, записывающий результаты экзекуции программы:

```

ofkhayjdarovdk5n18@fedora:~$ cat result.txt
1:Apple - это фрукт
3:Apple iPhone - смартфон
5:apple pie - яблочный пирог
ofkhayjdarovdk5n18@fedora:~$ █

```

Реестр-файл результата

Результат выполнения:

Скрипт корректно распознает флаги. Если передать ключ -С, поиск становится чувствительным к регистру. Если передать -п, в результат добавляются номера строк.

2.1 Взаимодействие Си и Bash

Условие: Написать программу на Си, которая определяет знак числа и завершается с кодом 0, 1 или 2. Написать скрипт, который вызывает эту программу и анализирует код возврата через \$?.

Создаем файл:

```
ofkhayjdarovdk5n18@fedora:~$ nano check_num.c
ofkhayjdarovdk5n18@fedora:~$
```

Создание файла

Пишем код программы, которая проверяет вводимые числа:

```
GNU nano 8.5                                check_num.c                                Modified
#include <stdio.h>
#include <stdlib.h>

int main() {
    int num;
    printf("Введите число: ");

    // Проверяем, что введено именно число
    if (scanf("%d", &num) != 1) {
        return 3; // Код ошибки ввода
    }

    if (num > 0) return 1; // Положительное
    else if (num < 0) return 2; // Отрицательное
    else return 0; // Равно нулю
}
```

Программа проверки чисел

Далее код для компиляция Си-программы:

```
ofkhayjdarovdk5n18@fedora:~$ nano task2_wrapper.sh
ofkhayjdarovdk5n18@fedora:~$ chmod +x task2_wrapper.sh
ofkhayjdarovdk5n18@fedora:~$
```

Создание файла

```

GNU nano 8.5
#!/bin/bash

echo "🔧 Компиляция С-программы..."
gcc check_num.c -o check_num

# Проверяем, успешно ли прошла компиляция
if [ $? -ne 0 ]; then
    echo "❌ Ошибка компиляции! Проверьте синтаксис С-кода."
    exit 1
fi

echo "✅ Компиляция успешна."
echo ""
echo "🚀 Запуск С-программы:"
./check_num

# Мгновенно сохраняем код завершения в переменную
STATUS=$?

echo ""
echo "📄 Код завершения программы: $STATUS"
case $STATUS in
    0) echo "🟢 Введённое число равно НУЛЮ" ;;
    1) echo "📈 Введённое число ПОЛОЖИТЕЛЬНОЕ" ;;
    2) echo "📉 Введённое число ОТРИЦАТЕЛЬНОЕ" ;;
    3) echo "⚠️ Ошибка: введено не число!" ;;
    *) echo "❓ Неизвестный код завершения: $STATUS" ;;
esac

```

Си-программа

Далее делаем ее исполняемой через команду `chmod +x task2_wrapper.sh` (из первого скрина)

Запуск программы:

```

ofkhayjdarovdk5n18@fedora:~$ ./task2_wrapper.sh
🔧 Компиляция С-программы...
✅ Компиляция успешна.

🚀 Запуск С-программы:
Введите число: 5

📊 Код завершения программы: 1
📌 Введённое число ПОЛОЖИТЕЛЬНОЕ
ofkhayjdarovdk5n18@fedora:~$ ./task2_wrapper.sh
🔧 Компиляция С-программы...
✅ Компиляция успешна.

🚀 Запуск С-программы:
Введите число: -3

📊 Код завершения программы: 2
📌 Введённое число ОТРИЦАТЕЛЬНОЕ
ofkhayjdarovdk5n18@fedora:~$ ./task2_wrapper.sh
🔧 Компиляция С-программы...
✅ Компиляция успешна.

🚀 Запуск С-программы:
Введите число: 0

📊 Код завершения программы: 0
🟢 Введённое число равно НУЛЮ
ofkhayjdarovdk5n18@fedora:~$

```

Запуск программы

– проверяем, ввода Положительное, Отрицательное числа, и в конце вводим число 0 – на что программа успешно отвечает нам и определила их соответствующе

Результат выполнения:

Скрипт компилирует .с файл, запускает его, считывает введенное число и выводит вердикт на основе кода завершения программы Си.

2.2 Циклы и управление файлами

Условие: Написать файл, создающий N файлов (1.tmp...N.tmp) по аргументу. Скрипт должен уметь также и удалять эти файлы.

Создаем файл:

```

ofkhayjdarovdk5n18@fedora:~$ nano task3_files.sh
ofkhayjdarovdk5n18@fedora:~$ chmod +x task3_files.sh
ofkhayjdarovdk5n18@fedora:~$

```

Создание файла

Потом вводим программу, которая создаем N файлом и удаляет также эти файлы:

```
GNU nano 8.5
#!/bin/bash

# =====
# Лабораторная работа №11, Задание 3
# Создание и удаление файлов циклом
# =====

# Считываем аргументы
ACTION=$1 # Действие (create или delete)
N=$2      # Количество файлов

# Проверка: переданы ли оба аргумента
if [ -z "$ACTION" ] || [ -z "$N" ]; then
    echo "❌ Ошибка: укажите действие и количество файлов"
    echo "Использование: $0 <create|delete> <N>"
    echo "Пример: $0 create 5"
    exit 1
fi

# Проверка: является ли N числом
if ! [[ "$N" =~ ^[0-9]+$ ]]; then
    echo "❌ Ошибка: N должно быть целым числом"
    exit 1
fi

echo "Начинаю выполнение действия: $ACTION для $N файлов..."

case $ACTION in
    create)
        echo "Режим создания:"
        for ((i=1; i<=N; i++)); do
            touch "${i}.tmp"
            echo "✅ Создан файл: ${i}.tmp"
        done
        ;;
    delete)
        echo "🗑️ Режим удаления:"
        for ((i=1; i<=N; i++)); do

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\ Replace	^U Paste	^J Justify

Создание программы

```
GNU nano 8.5
exit 1
fi

# Проверка: является ли N числом
if ! [[ "$N" =~ ^[0-9]+$ ]]; then
    echo "❌ Ошибка: N должно быть целым числом"
    exit 1
fi

echo "Начинаю выполнение действия: $ACTION для $N файлов..."

case $ACTION in
    create)
        echo "Режим создания:"
        for ((i=1; i<=N; i++)); do
            touch "${i}.tmp"
            echo "✅ Создан файл: ${i}.tmp"
        done
        ;;
    delete)
        echo "🗑️ Режим удаления:"
        for ((i=1; i<=N; i++)); do
            if [ -f "${i}.tmp" ]; then
                rm "${i}.tmp"
                echo "🗑️ Удален файл: ${i}.tmp"
            else
                echo "Файл ${i}.tmp не найден (пропуск)"
            fi
        done
        ;;
    *)
        echo "❌ Неизвестное действие '$ACTION'. Используйте 'create' или 'delete'"
        exit 1
        ;;
esac

echo "✅ Готово!"
```

GNU nano 8.5 keyboard shortcuts: ^G Help, ^O Write Out, ^F Where Is, ^K Cut, ^T Execute, ^X Exit, ^R Read File, ^\ Replace, ^U Paste, ^J Justify

Создание программы

Делаем её исполняемой с помощью команды `chmod +x` (из первого скрина)

Запускаем и проверяем работу файла - успешно было создано 5 файлов

```
ofkhayjdarovdk5n18@fedora:~$ ./task3_files.sh crate 5
Начинаю выполнение действия: crate для 5 файлов...
❌ Неизвестное действие 'crate'. Используйте 'create' или 'delete'
ofkhayjdarovdk5n18@fedora:~$ ./task3_files.sh create 5
Начинаю выполнение действия: create для 5 файлов...
Режим создания:
✅ Создан файл: 1.tmp
✅ Создан файл: 2.tmp
✅ Создан файл: 3.tmp
✅ Создан файл: 4.tmp
✅ Создан файл: 5.tmp
✅ Готово!
ofkhayjdarovdk5n18@fedora:~$
```

Запуск работы файла

Проверяем их наличие с помощью команды ls

```
ofkhayjdarovdk5n18@fedora:~$ ls -l *.tmp
-rw-r--r--. 1 ofkhayjdarovdk5n18 ofkhayjdarovdk5n18 0 May  9 17:53 1.tmp
-rw-r--r--. 1 ofkhayjdarovdk5n18 ofkhayjdarovdk5n18 0 May  9 17:53 2.tmp
-rw-r--r--. 1 ofkhayjdarovdk5n18 ofkhayjdarovdk5n18 0 May  9 17:53 3.tmp
-rw-r--r--. 1 ofkhayjdarovdk5n18 ofkhayjdarovdk5n18 0 May  9 17:53 4.tmp
-rw-r--r--. 1 ofkhayjdarovdk5n18 ofkhayjdarovdk5n18 0 May  9 17:53 5.tmp
ofkhayjdarovdk5n18@fedora:~$
```

Проверка наличия файлов

– как видим, да, файлы имеются

Далее удаляем их

```
ofkhayjdarovdk5n18@fedora:~$ ./task3_files.sh delete 5
Начинаю выполнение действия: delete для 5 файлов...
🗑 Режим удаления:
🗑 Удален файл: 1.tmp
🗑 Удален файл: 2.tmp
🗑 Удален файл: 3.tmp
🗑 Удален файл: 4.tmp
🗑 Удален файл: 5.tmp
✅ Готово!
ofkhayjdarovdk5n18@fedora:~$
```

Удаление файлов

Программа работает, мы успешно создали и удалили 5 файлов .tmp - формата

Результат выполнения:

Запуск ./task3_files.sh create 5 создает файлы 1.tmp...5.tmp. Запуск ./task3_files.sh delete 5 удаляет их.

2.3 Архивация с помощью find

Условие: Написать файл, который архивирует файлы, измененные менее недели назад, используя find и tar.

Создаем файл [Создание файла] :

```
ofkhayjdarovdk5n18@fedora:~$ nano task4_archive.sh
ofkhayjdarovdk5n18@fedora:~$
```

Создание файла

Далее пишем код программы, соответствующей условиям:

```

GNU nano 8.5
#!/bin/bash

# =====
# Лабораторная работа №11, Задание 4
# Архивация файлов, изменённых менее недели назад
# =====

# Проверяем, передан ли путь к директории
if [ -z "$1" ]; then
    echo "❌ Ошибка: укажите путь к директории"
    echo "Использование: $0 <путь_к_папке>"
    exit 1
fi

TARGET_DIR="$1"

# Проверяем существование директории
if [ ! -d "$TARGET_DIR" ]; then
    echo "❌ Ошибка: директория '$TARGET_DIR' не найдена!"
    exit 1
fi

ARCHIVE_NAME="week_backup.tar.gz"

echo "🔍 Ищем файлы в '$TARGET_DIR', изменённые менее 7 дней назад..."

# find флаги:
# -maxdepth 1 : искать только в текущей папке (не в подпапках)
# -type f      : только файлы (не папки)
# -mtime -7    : время изменения < 7 суток
FILE_LIST=$(find "$TARGET_DIR" -maxdepth 1 -type f -mtime -7)

# Если ничего не найдено
if [ -z "$FILE_LIST" ]; then
    echo "⚠️ Файлов за последнюю неделю не найдено."
    exit 0
fi

```

Код программы, 1-я часть

```

        exit 1
    fi

    ARCHIVE_NAME="week_backup.tar.gz"

    echo "🔍 Ищем файлы в '$TARGET_DIR', изменённые менее 7 дней назад..."

    # find флаги:
    # -maxdepth 1 : искать только в текущей папке (не в подпапках)
    # -type f      : только файлы (не папки)
    # -mtime -7    : время изменения < 7 суток
    FILE_LIST=$(find "$TARGET_DIR" -maxdepth 1 -type f -mtime -7)

    # Если ничего не найдено
    if [ -z "$FILE_LIST" ]; then
        echo "⚠️ Файлов за последнюю неделю не найдено."
        exit 0
    fi

    echo "📁 Найденные файлы:"
    echo "$FILE_LIST"
    echo ""

    # Создаём архив
    # tar -czf : создать gz-архив
    # $FILE_LIST : список найденных файлов
    tar -czf "$ARCHIVE_NAME" $FILE_LIST

    # Проверяем успех
    if [ $? -eq 0 ]; then
        echo "✅ Архив успешно создан: $ARCHIVE_NAME"
        echo "📄 Размер архива:"
        ls -lh "$ARCHIVE_NAME"
    else
        echo "❌ Ошибка при создании архива!"
        exit 1
    fi
fi

```

Код программы, 2-я часть

Далее делаем файл исполняемым [Файл исполняемый] (с помощью команды `chmod +x`):

```

ofkhayjdarovdk5n18@fedora:~$ chmod +x task4_archive.sh
ofkhayjdarovdk5n18@fedora:~$

```

Файл исполняемый

Создаем файлы-реестры, с давностью изменения 2 дня назад, сегодня, 10 дней назад и 3 дня назад [Файлы-реестры] :

```

ofkhayjdarovdk5n18@fedora:~$ touch -d "2 days ago" recent_doc.txt
ofkhayjdarovdk5n18@fedora:~$ touch today_notes.txt
ofkhayjdarovdk5n18@fedora:~$ touch -d "10 days ago" old_archive.zip
ofkhayjdarovdk5n18@fedora:~$ touch -d "3 days ago" project_draft.pdf

```

Файлы-реестры

Запуск работы и [Результат] :

```

ofkhayjdardvk5n18@fedora:~$ ./task4_archive.sh .
Ищем файлы в '.', изменённые менее 7 дней назад...
Найденные файлы:
./vboxclient-clipboard-tty2-control.pid
./vboxclient-seamless-tty2-control.pid
./vboxclient-draganddrop-tty2-control.pid
./vboxclient-hostversion-tty2-control.pid
./bash_history
./vboxclient-clipboard-tty2-service.pid
./task1_search.sh
./test_data.txt
./result.txt
./check_num.c
./task2_wrapper.sh
./check_num
./task3_files.sh
./task4_archive.sh
./recent_doc.txt
./today_notes.txt
./project_draft.pdf

✓ Архив успешно создан: week_backup.tar.gz
Размер архива:
-rw-r--r--. 1 ofkhayjdardvk5n18 ofkhayjdardvk5n18 9.4K May  9 18:02 week_backup.tar.gz
ofkhayjdardvk5n18@fedora:~$

```

Результат

Результат выполнения:

Скрипт находит файлы (например, recent.txt, созданный вчера) и добавляет их в week_backup.tar.gz. Файлы, созданные месяц назад, игнорируются.

Выводы

В ходе лабораторной работы №13 я научился и изучил:

1. Синтаксический анализ командной строки с помощью команды getopt и реализовал поиск через grep;
 2. Изучил механизм взаимодействия интерпретатора Bash и компилируемых программ (Си) через код завершения \$?;
 3. Научился использовать циклы for для пакетного создания и удаления файлов ;
 4. Применил утилиту find с условием по времени изменения (-mtime) для выборочной архивации данных.
-

Контрольные вопросы

1. Каково предназначение команды `getopts`?

`getopts` предназначена для синтаксического анализа командной строки. Она позволяет скрипту корректно обрабатывать флаги (опции) и их аргументы, упрощая создание профессиональных утилит.

1. Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы (*, ?, []) используются командным процессором для “разворачивания” (globbing) шаблонов в списки существующих имён файлов.

Например, *.txt генерирует список всех текстовых файлов в директории.

1. Какие операторы управления действиями вы знаете?

Операторы:

Последовательности: ; Логические: && (И), || (ИЛИ) Конвейер: |
Условные: if, case Циклы: for, while, until

1. Какие операторы используются для прерывания цикла?

`break` — полностью прекращает выполнение цикла и выходит из него. `continue` — прерывает текущую итерацию цикла и переходит к следующей проверке условия.

1. Для чего нужны команды `false` и `true`?

Эти команды используются для создания условий в циклах и проверках:

`true` всегда возвращает код успеха (0). `false` всегда возвращает код ошибки (1).

Они часто используются в бесконечных циклах: `while true; do ...; done`.

1. Что означает строка `if test -f mans/i.$s?`

Эта строка проверяет существование обычного файла по пути `mans/i.$s`.

Оператор `test -f` возвращает “истину” (0), если файл существует и является обычным файлом (не директорией).

1. Объясните различия между конструкциями `while` и `until`.

`while` выполняет тело цикла пока условие истинно (код возврата 0). `until` выполняет тело цикла пока условие ложно (код возврата не 0), т.е. пока условие не станет истинным.

Список литературы

Список литературы

1. Кулябов Д. С. и др. Операционные системы: учебное пособие. — М.: РУДН, 2024. — С. 97–98.
2. GNU Bash Manual. Free Software Foundation, 2026. — URL: <https://www.gnu.org/software/bash/manual/>